

4.1: SQL und Relationenalgebra

a) Relationale Algebra: Kunden jünger als 40 Jahre

$$\pi_{k_name}(\sigma_{k_alter < 40}(kunde))$$

b) Relationale Algebra: Bankkonten mit hohem Darlehen und Kontaktperson nicht in Berlin

$$\pi_{b_id}(\sigma_{d_kredithoehe > 30000}(darlehen) \bowtie_{d_bankkonto=b_id} bankkonto \bowtie_{b_kontaktperson=m_id} mitarbeiter \bowtie_{m_adresse \neq Berlin} adresse)$$

c) SQL: Name, Alter, PLZ aller Kunden, aufsteigend sortiert nach Alter

```
1 SELECT kunde.k_name, kunde.k_alter, adresse.a_plz
2 FROM kunde, adresse
3 WHERE kunde.k_adresse = adresse.a_id
4 ORDER BY kunde.k_alter ASC;
```

d) SQL: Mitarbeiter mit mehr als 10 qualifizierten Bankkonten

```
1 SELECT mitarbeiter.m_name, COUNT(bankkonto.b_id)
2 FROM mitarbeiter, bankkonto
3 WHERE mitarbeiter.m_id = bankkonto.b_kontaktperson
4       AND bankkonto.b_guthaben > 20000
5 GROUP BY mitarbeiter.m_id, mitarbeiter.m_name
6 HAVING COUNT(bankkonto.b_id) > 10;
```

e) SQL: Kunden mit Darlehen über dem Durchschnitt

```
1 SELECT DISTINCT kunde.k_name, kunde.k_alter
2 FROM kunde, darlehen
3 WHERE kunde.k_id = darlehen.d_kunde
4       AND darlehen.d_kredithoehe > (
5         SELECT AVG(darlehen.d_kredithoehe)
6         FROM darlehen
7       );
```

f) SQL: Adressen-Ortsnamen standardisieren ('Koeln...')

```
1 UPDATE adresse
2 SET a_ort = 'Koeln'
3 WHERE a_ort LIKE 'Koeln%';
```

Abgabe 4.2